

# PacCafe Database Data Check

Notebook ini digunakan untuk mengecek data yang ada di masing-masing database pada project `mentoring_2`:

- Source database: `paccafe`
- Staging database: `staging`
- Warehouse database: `warehouse`
- Log database: `pipeline_log`

Notebook ini memakai command-line `psql` dari Python `subprocess`, sehingga tidak membutuhkan package tambahan seperti `psycopg`, `sqlalchemy`, atau `pandas`. Notebook dijalankan dari environment `uv` project.

Scope saat ini sudah end-to-end: PostgreSQL source tables dan copied spreadsheet data `store_branch_paccoffee.csv` masuk ke staging lalu warehouse. Expected count untuk `store_branch` dan `dim_store_branch` adalah `3`.

## 1. Pre-check

Pastikan dependency notebook dan service Docker sudah siap. Jalankan dari root `mentoring_2`:

```
uv sync --group notebook
cd data_pipeline_paccafe
docker compose up -d
cd ..
```

Jika ingin refresh data sebelum inspeksi, jalankan pipeline dulu:

```
uv run paccafe-pipeline
```

Lalu buka notebook ini dengan:

```
uv run jupyter notebook notebooks/db_data_check.ipynb
```

```
In [1]: import csv
import io
import os
import shutil
import subprocess
from IPython.display import Markdown, display

psql_path = shutil.which("psql")
if not psql_path:
    raise RuntimeError("psql is not available on PATH. Install PostgreSQL client")

print(f"psql found: {psql_path}")
```

psql found: /opt/homebrew/bin/psql

## 2. Database Configuration

Default value mengikuti `.env` yang dipakai Docker Compose.

```
In [2]: DB_CONFIGS = {
    "source": {
        "database": os.getenv("SRC_POSTGRES_DB", "paccafe"),
        "host": os.getenv("SRC_POSTGRES_HOST", "localhost"),
        "port": os.getenv("SRC_POSTGRES_PORT", "5433"),
        "user": os.getenv("SRC_POSTGRES_USER", "postgres"),
        "password": os.getenv("SRC_POSTGRES_PASSWORD", "postgres"),
    },
    "staging": {
        "database": os.getenv("STG_POSTGRES_DB", "staging"),
        "host": os.getenv("STG_POSTGRES_HOST", "localhost"),
        "port": os.getenv("STG_POSTGRES_PORT", "5434"),
        "user": os.getenv("STG_POSTGRES_USER", "postgres"),
        "password": os.getenv("STG_POSTGRES_PASSWORD", "postgres"),
    },
    "warehouse": {
        "database": os.getenv("WH_POSTGRES_DB", "warehouse"),
        "host": os.getenv("WH_POSTGRES_HOST", "localhost"),
        "port": os.getenv("WH_POSTGRES_PORT", "5435"),
        "user": os.getenv("WH_POSTGRES_USER", "postgres"),
        "password": os.getenv("WH_POSTGRES_PASSWORD", "postgres"),
    },
    "log": {
        "database": os.getenv("LOG_POSTGRES_DB", "pipeline_log"),
        "host": os.getenv("LOG_POSTGRES_HOST", "localhost"),
        "port": os.getenv("LOG_POSTGRES_PORT", "5436"),
        "user": os.getenv("LOG_POSTGRES_USER", "postgres"),
        "password": os.getenv("LOG_POSTGRES_PASSWORD", "postgres"),
    },
}

def db_url(db_key):
    cfg = DB_CONFIGS[db_key]
    return f"postgresql://{cfg['user']}:{cfg['password']}@{cfg['host']}:{cfg['port']}/{cfg['database']}"

for name, cfg in DB_CONFIGS.items():
    print(f"{name}: {cfg['host']}:{cfg['port']}/{cfg['database']}")
```

source: localhost:5433/paccafe  
 staging: localhost:5434/staging  
 warehouse: localhost:5435/warehouse  
 log: localhost:5436/pipeline\_log

## 3. Helper Functions

```
In [3]: def run_psql(db_key, sql, *, csv_output=True):
    if csv_output:
```

```

        sql = f"COPY ({sql}) TO STDOUT WITH CSV HEADER"
    command = [
        "psql",
        db_url(db_key),
        "-X",
        "-q",
        "-v",
        "ON_ERROR_STOP=1",
        "-c",
        sql,
    ]
    result = subprocess.run(command, text=True, capture_output=True, check=False)
    if result.returncode != 0:
        raise RuntimeError(result.stderr.strip() or result.stdout.strip())
    return result.stdout

def fetch_rows(db_key, sql):
    output = run_psql(db_key, sql, csv_output=True)
    return list(csv.DictReader(io.StringIO(output)))

def quote_identifier(identifier):
    if not identifier.replace("_", "").isalnum():
        raise ValueError(f"Unsafe identifier: {identifier!r}")
    return "'" + identifier.replace("'", '""') + "'"

def markdown_table(rows, max_rows=20):
    if not rows:
        return "_No rows returned._"
    rows = rows[:max_rows]
    headers = list(rows[0].keys())
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join(["----"] * len(headers))]
    for row in rows:
        values = [str(row.get(header, "")).replace("\n", " ") for header in headers]
        lines.append("| " + " | ".join(values) + " |")
    return "\n".join(lines)

def show_rows(rows, title=None, max_rows=20):
    if title:
        display(Markdown(f"### {title}"))
    display(Markdown(markdown_table(rows, max_rows=max_rows)))

def list_tables(db_key):
    return fetch_rows(
        db_key,
        """
        SELECT table_schema, table_name
        FROM information_schema.tables
        WHERE table_schema = 'public'
          AND table_type = 'BASE TABLE'
        ORDER BY table_name
        """,
    )

```

```

    )

def row_counts(db_key):
    tables = list_tables(db_key)
    results = []
    for table in tables:
        table_name = table["table_name"]
        count_rows = fetch_rows(db_key, f"SELECT count(*) AS row_count FROM {table_name}")
        results.append({"table_name": table_name, "row_count": count_rows[0]})
    return results

def sample_table(db_key, table_name, limit=5):
    return fetch_rows(db_key, f"SELECT * FROM public.{quote_identifier(table_name)} LIMIT {limit}")

```

## 4. Connection Check

```

In [4]: connection_rows = []
        for db_key in DB_CONFIGS:
            rows = fetch_rows(
                db_key,
                """
                SELECT
                    current_database() AS database,
                    current_user AS user_name,
                    inet_server_port() AS server_port
                """
            )
            connection_rows.append({"db_key": db_key, **rows[0]})

        show_rows(connection_rows, "Connection Result")

```

### Connection Result

| db_key    | database     | user_name | server_port |
|-----------|--------------|-----------|-------------|
| source    | pacafe       | postgres  | 5432        |
| staging   | staging      | postgres  | 5432        |
| warehouse | warehouse    | postgres  | 5432        |
| log       | pipeline_log | postgres  | 5432        |

## 5. List Tables per Database

```

In [5]: for db_key in DB_CONFIGS:
        show_rows(list_tables(db_key), f"Tables in {db_key}")

```

### Tables in source

| table_schema | table_name         |
|--------------|--------------------|
| public       | customers          |
| public       | employees          |
| public       | inventory_tracking |
| public       | order_details      |
| public       | orders             |
| public       | products           |

## Tables in staging

| table_schema | table_name         |
|--------------|--------------------|
| public       | customers          |
| public       | employees          |
| public       | inventory_tracking |
| public       | order_details      |
| public       | orders             |
| public       | products           |
| public       | store_branch       |

## Tables in warehouse

| table_schema | table_name       |
|--------------|------------------|
| public       | dim_customers    |
| public       | dim_date         |
| public       | dim_employees    |
| public       | dim_products     |
| public       | dim_store_branch |
| public       | fct_inventory    |
| public       | fct_order        |

## Tables in log

| table_schema | table_name |
|--------------|------------|
| public       | etl_log    |

# 6. Row Count per Database

```
In [6]: all_counts = {}  
for db_key in DB_CONFIGS:  
    counts = row_counts(db_key)  
    all_counts[db_key] = counts  
    show_rows(counts, f"Row Counts in {db_key}")
```

## Row Counts in source

| table_name         | row_count |
|--------------------|-----------|
| customers          | 204       |
| employees          | 103       |
| inventory_tracking | 162       |
| order_details      | 3022      |
| orders             | 1010      |
| products           | 54        |

## Row Counts in staging

| table_name         | row_count |
|--------------------|-----------|
| customers          | 204       |
| employees          | 103       |
| inventory_tracking | 162       |
| order_details      | 3022      |
| orders             | 1010      |
| products           | 54        |
| store_branch       | 3         |

## Row Counts in warehouse

| table_name       | row_count |
|------------------|-----------|
| dim_customers    | 204       |
| dim_date         | 29220     |
| dim_employees    | 103       |
| dim_products     | 54        |
| dim_store_branch | 3         |
| fct_inventory    | 162       |
| fct_order        | 1010      |

## Row Counts in log

| table_name | row_count |
|------------|-----------|
| etl_log    | 64        |

## 7. Source Database Sample Data

```
In [7]: for table_name in ["customers", "employees", "orders", "order_details", "products":
        show_rows(sample_table("source", table_name, limit=5), f"source.public.{table_name}")
```

### source.public.customers

| customer_id | first_name | last_name | email                    | phone         | loyalty |
|-------------|------------|-----------|--------------------------|---------------|---------|
| 492         | Paul       | Hawkins   | bmorris@example.net      | 9958687561434 | 708     |
| 493         | Jessica    | Hendricks | sandramalone@example.com | 3378843199040 | 241     |
| 494         | Christian  | Shepherd  | lreed@example.net        | 8814427344096 | 239     |
| 495         | Robert     | Perry     | ninalucero@example.com   | 6819873900353 | 956     |
| 496         | Courtney   | Miller    | coxmartin@example.net    | 7185028607446 | 677     |

### source.public.employees

| employee_id | first_name | last_name | hire_date  | role     | email                      | city |
|-------------|------------|-----------|------------|----------|----------------------------|------|
| 1           | John       | Gordon    | 2020-07-19 | Cashier  | john04@example.net         | 2000 |
| 2           | Gregory    | Curry     | 2023-05-15 | Waitress | rachaelfreeman@example.org | 2000 |
| 3           | Chris      | Allen     | 2022-11-27 | Waitress | petercox@example.org       | 2000 |
| 4           | Ryan       | Mitchell  | 2021-07-06 | Cashier  | smithsteven@example.net    | 2000 |
| 5           | Tyler      | Wiggins   | 2021-10-21 | Manager  | vgriffin@example.com       | 2000 |

### source.public.orders

| order_id | employee_id | customer_id | order_date          | total_amount | payment_method | order |
|----------|-------------|-------------|---------------------|--------------|----------------|-------|
| 1484     | 29          |             | 2023-12-04 15:41:20 | 442.00       | Cash           | Comp  |
| 1798     | 66          | 542         | 2024-08-03 10:59:12 | 344.00       | Cash           | Comp  |
| 1489     | 71          | 571         | 2023-12-13 12:11:13 | 465.00       | Credit Card    | Comp  |
| 1269     | 71          |             | 2023-07-14 23:06:02 | 292.00       | Debit Card     | Comp  |
| 1989     | 57          | 680         | 2024-12-21 10:33:32 | 35.00        | Cash           | Comp  |

source.public.order\_details

| order_detail_id | order_id | product_id | quantity | unit_price | subtotal | created_at          |
|-----------------|----------|------------|----------|------------|----------|---------------------|
| 1               | 1001     | 80         | 1        | 15.00      | 15.00    | 2023-01-03 10:42:46 |
| 2               | 1001     | 65         | 8        | 18.00      | 144.00   | 2023-01-03 10:42:46 |
| 3               | 1002     | 57         | 4        | 43.00      | 172.00   | 2023-01-03 15:25:52 |
| 4               | 1002     | 58         | 2        | 9.00       | 18.00    | 2023-01-03 15:25:52 |
| 5               | 1002     | 69         | 6        | 41.00      | 246.00   | 2023-01-03 15:25:52 |

source.public.products

| product_id | product_name     | category | unit_price | cost_price | in_stock | created_at          |
|------------|------------------|----------|------------|------------|----------|---------------------|
| 53         | Pastry situation | Pastry   | \$47.00    | \$43.00    | t        | 2024-09-21 00:00:00 |
| 54         | Pastry sure      | Pastry   | \$47.00    | \$43.00    | t        | 2022-10-20 00:00:00 |
| 55         | Pastry statement | Pastry   | \$17.00    | \$14.00    | t        | 2024-01-17 00:00:00 |
| 56         | Salad on         | Salad    | \$21.00    | \$20.00    | t        | 2020-08-27 00:00:00 |
| 57         | Salad really     | Salad    | \$43.00    | \$40.00    | t        | 2020-05-08 00:00:00 |



source.public.inventory\_tracking

| tracking_id | product_id | quantity_change | change_date         | reason  | created_at          |
|-------------|------------|-----------------|---------------------|---------|---------------------|
| 1           | 53         | 3               | 2024-11-01 00:00:00 | Restock | 2024-11-01 00:00:00 |
| 2           | 54         | 10              | 2024-08-05 00:00:00 | Restock | 2024-08-05 00:00:00 |
| 3           | 55         | 6               | 2023-11-12 00:00:00 | Damaged | 2023-11-12 00:00:00 |
| 4           | 56         | 8               | 2023-05-29 00:00:00 | Damaged | 2023-05-29 00:00:00 |
| 5           | 57         | 6               | 2023-09-22 00:00:00 | Restock | 2023-09-22 00:00:00 |

8. Staging Database Sample Data

```
In [8]: for table_name in ["customers", "employees", "orders", "order_details", "products":
        show_rows(sample_table("staging", table_name, limit=5), f"staging.public.{table_name}")
```

staging.public.customers

| customer_id | first_name | last_name | email                    | phone         | loyalty |
|-------------|------------|-----------|--------------------------|---------------|---------|
| 492         | Paul       | Hawkins   | bmorris@example.net      | 9958687561434 | 708     |
| 493         | Jessica    | Hendricks | sandramalone@example.com | 3378843199040 | 241     |
| 494         | Christian  | Shepherd  | lreed@example.net        | 8814427344096 | 239     |
| 495         | Robert     | Perry     | ninalucero@example.com   | 6819873900353 | 956     |
| 496         | Courtney   | Miller    | coxmartin@example.net    | 7185028607446 | 677     |

staging.public.employees

| employee_id | first_name | last_name | hire_date  | role     | email                      | city |
|-------------|------------|-----------|------------|----------|----------------------------|------|
| 1           | John       | Gordon    | 2020-07-19 | Cashier  | john04@example.net         | 2000 |
| 2           | Gregory    | Curry     | 2023-05-15 | Waitress | rachaelfreeman@example.org | 2000 |
| 3           | Chris      | Allen     | 2022-11-27 | Waitress | petercox@example.org       | 2000 |
| 4           | Ryan       | Mitchell  | 2021-07-06 | Cashier  | smithsteven@example.net    | 2000 |
| 5           | Tyler      | Wiggins   | 2021-10-21 | Manager  | vgriffin@example.com       | 2000 |

staging.public.orders

| order_id | employee_id | customer_id | order_date          | total_amount | payment_method | order_status |
|----------|-------------|-------------|---------------------|--------------|----------------|--------------|
| 1484     | 29          |             | 2023-12-04 15:41:20 | 442.00       | Cash           | Completed    |
| 1798     | 66          | 542         | 2024-08-03 10:59:12 | 344.00       | Cash           | Completed    |
| 1489     | 71          | 571         | 2023-12-13 12:11:13 | 465.00       | Credit Card    | Completed    |
| 1269     | 71          |             | 2023-07-14 23:06:02 | 292.00       | Debit Card     | Completed    |
| 1989     | 57          | 680         | 2024-12-21 10:33:32 | 35.00        | Cash           | Completed    |

staging.public.order\_details

| order_detail_id | order_id | product_id | quantity | unit_price | subtotal | created_at          |
|-----------------|----------|------------|----------|------------|----------|---------------------|
| 1               | 1001     | 80         | 1        | 15.00      | 15.00    | 2023-01-03 10:42:46 |
| 2               | 1001     | 65         | 8        | 18.00      | 144.00   | 2023-01-03 10:42:46 |
| 3               | 1002     | 57         | 4        | 43.00      | 172.00   | 2023-01-03 15:25:52 |
| 4               | 1002     | 58         | 2        | 9.00       | 18.00    | 2023-01-03 15:25:52 |
| 5               | 1002     | 69         | 6        | 41.00      | 246.00   | 2023-01-03 15:25:52 |

staging.public.products

| product_id | product_name     | category | unit_price | cost_price | in_stock | store_branch | cr                  |
|------------|------------------|----------|------------|------------|----------|--------------|---------------------|
| 53         | Pastry situation | Pastry   | \$47.00    | \$43.00    | t        |              | 2023-01-03 10:42:46 |
| 54         | Pastry sure      | Pastry   | \$47.00    | \$43.00    | t        |              | 2023-01-03 10:42:46 |
| 55         | Pastry statement | Pastry   | \$17.00    | \$14.00    | t        |              | 2023-01-03 15:25:52 |
| 56         | Salad on         | Salad    | \$21.00    | \$20.00    | t        |              | 2023-01-03 15:25:52 |
| 57         | Salad really     | Salad    | \$43.00    | \$40.00    | t        |              | 2023-01-03 15:25:52 |

staging.public.inventory\_tracking

| tracking_id | product_id | quantity_change | change_date         | reason  | created_at          |
|-------------|------------|-----------------|---------------------|---------|---------------------|
| 1           | 53         | 3               | 2024-11-01 00:00:00 | Restock | 2024-11-01 00:00:00 |
| 2           | 54         | 10              | 2024-08-05 00:00:00 | Restock | 2024-08-05 00:00:00 |
| 3           | 55         | 6               | 2023-11-12 00:00:00 | Damaged | 2023-11-12 00:00:00 |
| 4           | 56         | 8               | 2023-05-29 00:00:00 | Damaged | 2023-05-29 00:00:00 |
| 5           | 57         | 6               | 2023-09-22 00:00:00 | Restock | 2023-09-22 00:00:00 |

staging.public.store\_branch

| store_id | store_name     | created_at          |
|----------|----------------|---------------------|
| 1        | Dapur Kenangan | 2025-01-31 20:43:18 |
| 2        | Laci Coffee    | 2025-01-31 20:43:18 |
| 3        | Setara Coffee  | 2025-01-31 20:43:18 |

9. Warehouse Database Sample Data

```
In [9]: for table_name in ["dim_customers", "dim_employees", "dim_products", "dim_st
        show_rows(sample_table("warehouse", table_name, limit=5), f"warehouse.pu
```

warehouse.public.dim\_customers

| sk_customer_id                       | nk_customer_id | first_name | last_name | email                    |    |
|--------------------------------------|----------------|------------|-----------|--------------------------|----|
| fe3dc8da-5f91-4b41-8ec8-d26e05db4146 | 492            | Paul       | Hawkins   | bmorris@example.net      | 99 |
| b027c644-f517-41c9-8064-7817776ac9b7 | 493            | Jessica    | Hendricks | sandramalone@example.com | 33 |
| 7120c76a-d6da-473e-bca9-f06c761d80c3 | 494            | Christian  | Shepherd  | lreed@example.net        | 88 |
| 7fba6e16-3f73-4b0b-b116-9b5a6edc7009 | 495            | Robert     | Perry     | ninalucero@example.com   | 68 |
| 3de37d4d-b305-468b-800a-82a7acf08d86 | 496            | Courtney   | Miller    | coxmartin@example.net    | 71 |

warehouse.public.dim\_employees

| sk_employee_id                       | nk_employee_id | first_name | last_name | hire_date | role     |            |
|--------------------------------------|----------------|------------|-----------|-----------|----------|------------|
| d116b82c-ca71-4731-9201-b210d4d1c465 | 1              | John       | Gordon    | 20200719  | Cashier  | john04@e   |
| 2f758202-2b0d-4eae-b871-b0d4aee4be81 | 2              | Gregory    | Curry     | 20230515  | Waitress | rachaelfre |
| 8b633d5b-8a85-4530-bf4b-2c17a659c7dc | 3              | Chris      | Allen     | 20221127  | Waitress | petercox@  |
| 04133d22-ba82-4319-8ba5-83295a16a3f1 | 4              | Ryan       | Mitchell  | 20210706  | Cashier  | smithsteve |
| d14e833d-a7b6-44c3-8448-6bddd9420631 | 5              | Tyler      | Wiggins   | 20211021  | Manager  | vgriffin@e |

## warehouse.public.dim\_products

| sk_product_id                        | nk_product_id | product_name         | category | unit_price | cost_price | in_stoc |
|--------------------------------------|---------------|----------------------|----------|------------|------------|---------|
| 68d95f49-4509-4a0f-a7fe-79ff61b12728 | 106           | Salad clearly        | Salad    | -11.00     | 14.00      | t       |
| 9f2a11d4-e4df-46b1-ba70-dd3c7a26c9df | 105           | Pastry news          | Pastry   | -15.00     | 18.00      | f       |
| ed0130f7-314c-4a39-8d2a-70f8eb67fe13 | 104           | Smoothie happy       | Smoothie | 37.00      | 43.00      | t       |
| 384fa8da-1ef5-40b9-aeb3-9b4d844f0061 | 103           | Sandwich about       | Sandwich | 19.00      | 27.00      | t       |
| ce6e6cf3-4d2b-445d-a6c9-5bd50be0a6d0 | 102           | Sandwich information | Sandwich | 38.00      | 32.00      | t       |

## warehouse.public.dim\_store\_branch

| sk_store_id                          | nk_store_id | store_name     | created_at          |
|--------------------------------------|-------------|----------------|---------------------|
| 39fca7d0-ef43-4aaf-80dd-8820a6ccd44c | 1           | Dapur Kenangan | 2025-01-31 20:43:18 |
| 0eac90ba-943d-4700-be82-a1f1600535d1 | 2           | Laci Coffee    | 2025-01-31 20:43:18 |
| 4a72daa6-f07c-4150-ab70-938a052defe2 | 3           | Setara Coffee  | 2025-01-31 20:43:18 |

## warehouse.public.fct\_order

| sk_order_id                          | nk_order_id | sk_employee_id                       | sk_customer_id                       | sk_product_id                        | order_dat |
|--------------------------------------|-------------|--------------------------------------|--------------------------------------|--------------------------------------|-----------|
| 30e83f78-7edf-4ccf-a1d8-59a7bfb2ca16 | 1484        | 2b0a3c5d-50e5-4983-8ec4-1711e53815f9 |                                      | c2b0ef14-6c64-4b3f-9b6c-ebafb9383f21 | 20231204  |
| 5669d3b4-eaab-4557-92af-f07c18c1ae3d | 1798        | 993601e5-9410-49b0-96b3-baa0f7722070 | 573b3480-f626-4965-a6c1-f13fe73601de | 446f22e9-0715-4e68-a683-49ed44d11a4f | 20240803  |
| a94b9110-6bba-4c08-9de7-32ffbf39fff5 | 1489        | 75866f98-3ad7-4d52-87ee-e203e8fee6ec | fbdc7484-23ba-4924-b65a-f08b2e13aec5 | 831af378-0da1-46a0-9706-43b11c2b4196 | 20231213  |
| e5a13d41-7f3f-442e-a8a1-45a33bf7d77b | 1269        | 75866f98-3ad7-4d52-87ee-e203e8fee6ec |                                      | 83717d0c-4a3e-4640-b262-5249ba359ccf | 20230714  |
| c997e1b0-e008-4d12-a464-9da09330a44a | 1989        | 6f8e2339-554a-4f65-a107-3a7657c34b39 | e9d13e13-5c76-4535-a591-89321688c8b0 | d4791fa5-fd86-4f84-b860-02ce65d026c7 | 20241221  |

## warehouse.public.fct\_inventory

| sk_tracking_id                       | nk_tracking_id | sk_product_id                        | quantity_change | change_date | reason  |
|--------------------------------------|----------------|--------------------------------------|-----------------|-------------|---------|
| 5c21f51e-04ca-42e2-a333-bff9d0aa6b1c | 1              | 124ef605-ba38-4e1e-8305-bd436ae2e84d | 3               | 20241101    | Restock |
| a7b1efc7-980d-4979-acc6-0dd2f652f2e8 | 2              | d8b88b35-9d6e-4d02-b6a8-5595e1a066bb | 10              | 20240805    | Restock |
| 08a13412-f207-42be-a7fd-8fcd29da4f51 | 3              | 83717d0c-4a3e-4640-b262-5249ba359ccf | 6               | 20231112    | Damaged |
| 1306e009-3e70-4951-99aa-1fe8d14c3010 | 4              | 831af378-0da1-46a0-9706-43b11c2b4196 | 8               | 20230529    | Damaged |
| 33216d9c-e6df-453d-bcde-48f3cc8f3020 | 5              | 7908f51b-8c7a-4d38-a876-3e74bdceb801 | 6               | 20230922    | Restock |

## 10. Log Database Sample Data

```
In [10]: show_rows(  
    fetch_rows(  
        "log",  
        ""  
        SELECT step, component, status, table_name, etl_date, error_msg  
        FROM public.etl_log  
        ORDER BY etl_date DESC  
        LIMIT 20  
        ""  
    ),  
    "Latest ETL Logs",  
    max_rows=20,  
)
```

### Latest ETL Logs

| step      | component                                           | status  | table_name         |
|-----------|-----------------------------------------------------|---------|--------------------|
| warehouse | load_warehouse_dimensions_and_facts                 | SUCCESS |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
|           |                                                     |         |                    |
| warehouse | load_warehouse_dimensions_and_facts                 | STARTED |                    |
| warehouse | sync_staging_tables_to_warehouse                    | SUCCESS |                    |
| warehouse | sync_staging_tables_to_warehouse:store_branch       | SUCCESS | store_branch       |
| warehouse | sync_staging_tables_to_warehouse:inventory_tracking | SUCCESS | inventory_tracking |
| warehouse | sync_staging_tables_to_warehouse:products           | SUCCESS | products           |
| warehouse | sync_staging_tables_to_warehouse:order_details      | SUCCESS | order_details      |
| warehouse | sync_staging_tables_to_warehouse:orders             | SUCCESS | orders             |
| warehouse | sync_staging_tables_to_warehouse:employees          | SUCCESS | employees          |
| warehouse | sync_staging_tables_to_warehouse:customers          | SUCCESS | customers          |
| warehouse | sync_staging_tables_to_warehouse                    | STARTED |                    |
| staging   | load_store_branch_spreadsheet_to_staging            | SUCCESS | store_branch       |
| staging   | load_store_branch_spreadsheet_to_staging            | STARTED | store_branch       |



| step    | component                          | status  | table_name         |
|---------|------------------------------------|---------|--------------------|
| staging | load_inventory_tracking_to_staging | SUCCESS | inventory_tracking |
| staging | load_inventory_tracking_to_staging | STARTED | inventory_tracking |
| staging | load_products_to_staging           | SUCCESS | products           |
| staging | load_products_to_staging           | STARTED | products           |
| staging | load_order_details_to_staging      | SUCCESS | order_details      |
| staging | load_order_details_to_staging      | STARTED | order_details      |
| staging | load_orders_to_staging             | SUCCESS | orders             |

## 11. Warehouse Quality Checks

```
In [11]: quality_checks = fetch_rows(
    "warehouse",
    """
    SELECT 'fct_order_bad_dates' AS check_name, count(*) AS issue_count
    FROM public.fct_order
    WHERE order_date NOT IN (SELECT date_id FROM public.dim_date)
    UNION ALL
    SELECT 'fct_inventory_bad_dates', count(*)
    FROM public.fct_inventory
    WHERE change_date NOT IN (SELECT date_id FROM public.dim_date)
    UNION ALL
    SELECT 'dim_customers_duplicate_nk', count(*)
    FROM (
        SELECT nk_customer_id
        FROM public.dim_customers
        GROUP BY nk_customer_id
        HAVING count(*) > 1
    ) duplicates
    UNION ALL
    SELECT 'dim_products_duplicate_nk', count(*)
    FROM (
        SELECT nk_product_id
        FROM public.dim_products
        GROUP BY nk_product_id
        HAVING count(*) > 1
    ) duplicates
    """,
)

show_rows(quality_checks, "Warehouse Quality Checks")
```

## Warehouse Quality Checks

| check_name                 | issue_count |
|----------------------------|-------------|
| fct_order_bad_dates        | 0           |
| fct_inventory_bad_dates    | 0           |
| dim_customers_duplicate_nk | 0           |
| dim_products_duplicate_nk  | 0           |

## 12. Notes

- `store_branch` dan `dim_store_branch` expected bernilai `3` karena data spreadsheet copy `store_branch_pacoffee.csv` sudah masuk pipeline.
- Notebook ini hanya untuk inspeksi data, bukan untuk menjalankan ETL.
- Untuk menjalankan ETL, gunakan salah satu command berikut dari root `mentoring_2`:

```
uv run paccafe-pipeline
```

atau:

```
uv run python -m paccafe_pipeline
```